Machine Learning

un fond bleu et apaisant

Type de Machine Learning

- Supervisé (on connaît la solution sur les données d'entraînement)
  - Régression
  - Classification

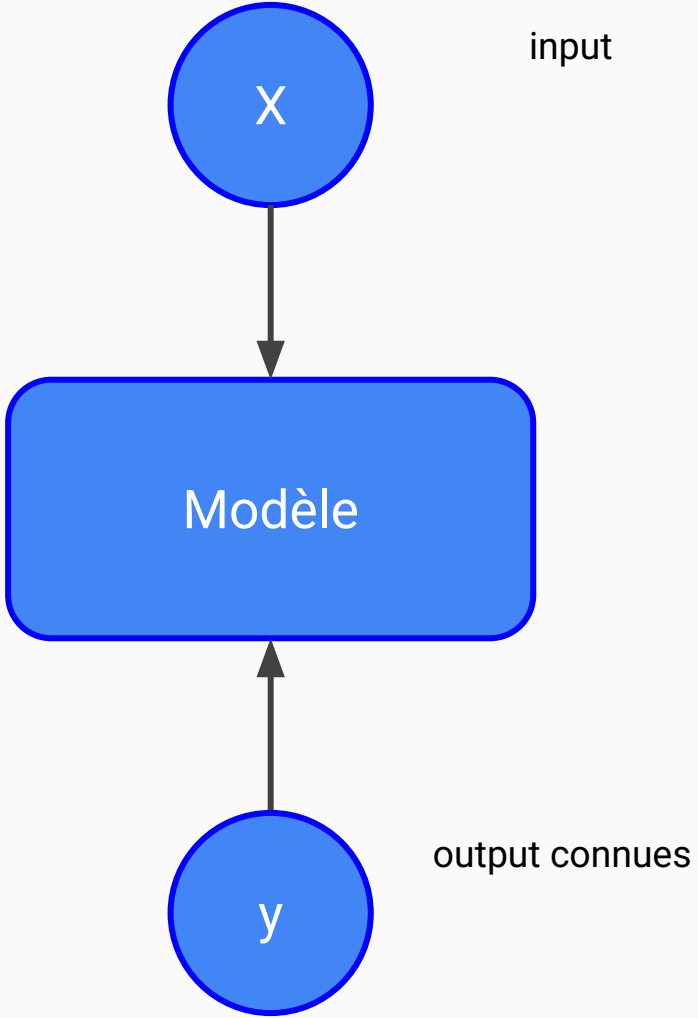
- Non Supervisé (on ne sait rien)
  - Clustering

Apprentissage supervisé

Supervisé

On connaît les données et on connaît les réponses!

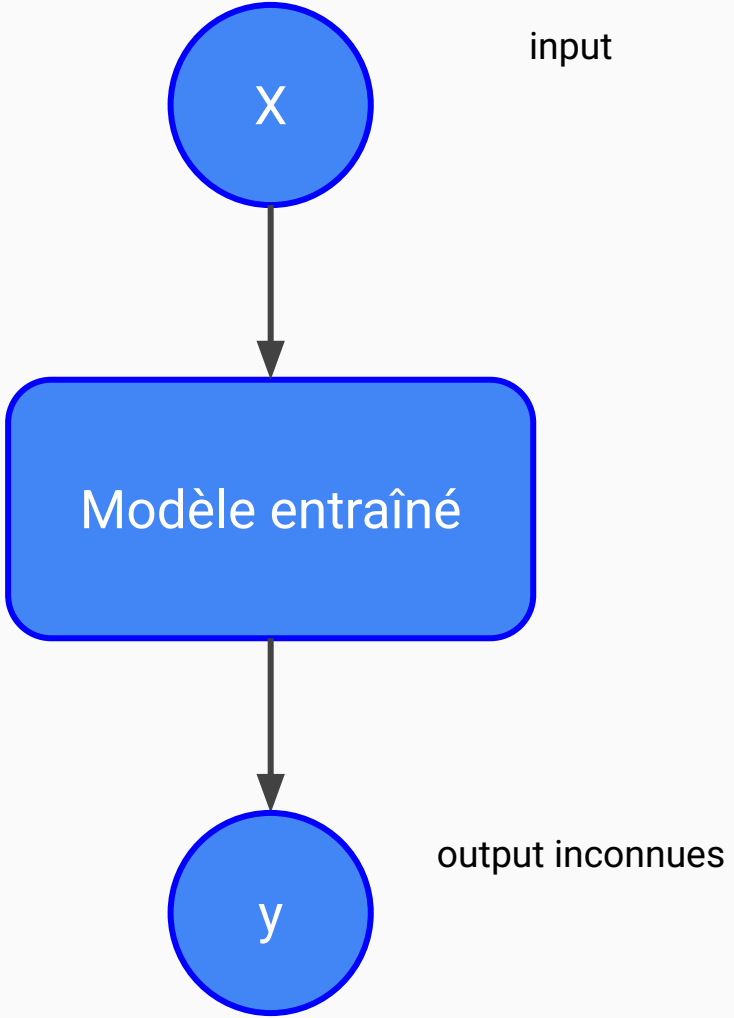
Le modèle sera optimisé pour trouver “y” en fonction de “X”



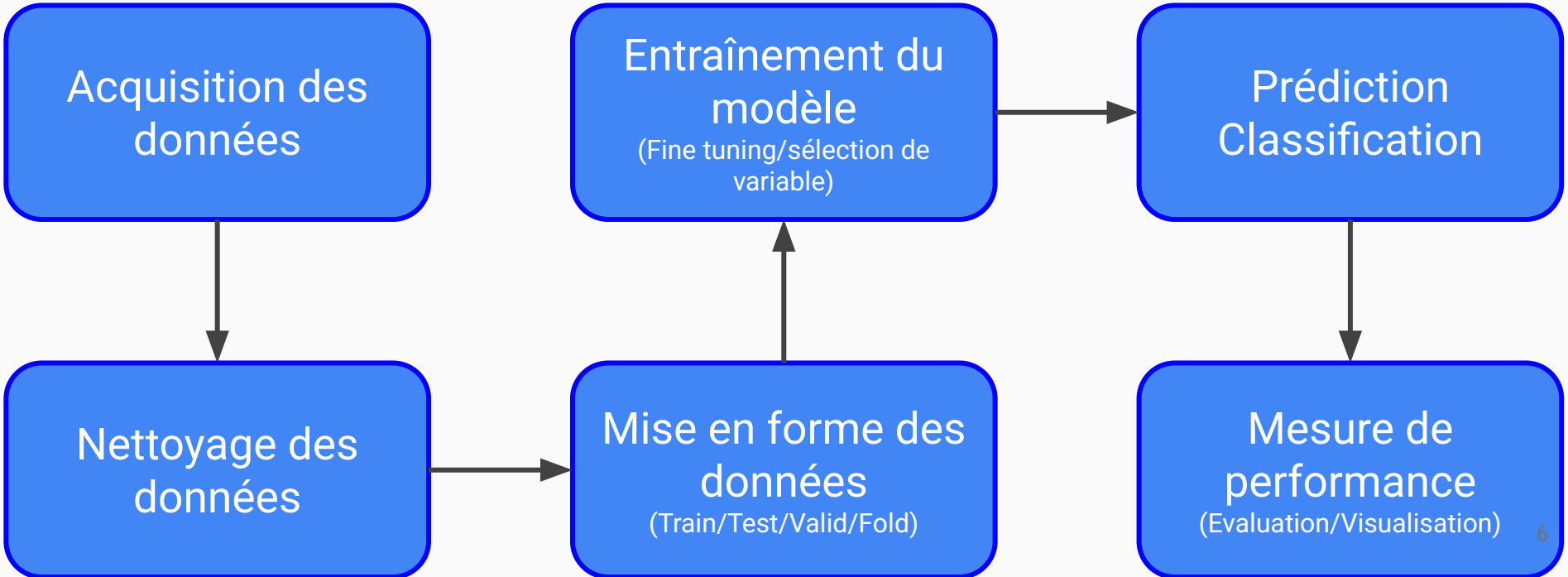
Supervisé

On connaît les données et on connaît les réponses!

Le modèle sera optimisé pour trouver “y” en fonction de “X”



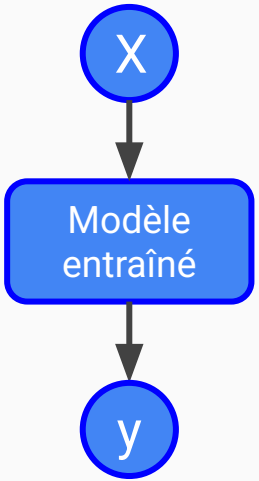
Pipeline Machine Learning (de base)



Régression Versus Classification

Régression

Prédire des valeurs réelles (Float)



Classification

Prédire une probabilité d'appartenance à une classe

y1	y2	y3	y4
3.141	2.718	1.618	6.626

chien	chat	poulpe	canard
20%	20%	10%	50%

Nettoyage des données

1. Explorer les données, afficher des graphiques.
2. Trouver les valeurs nulles ou aberrantes
    a. les enlever
    b. les mettres à la moyenne
3. Faire des choix, enlever une colonne qui contient plein de valeur manquante plutôt que d'enlever toutes les lignes qui peuvent avoir ces valeurs manquantes.
4. Modifier ou transformer les données (normalisation)
5. Transformer les données redondantes en données catégorielle

Explorer les données

Connaître le type des données et:

- le nombre de données et de colonnes. A quoi elles servent.
- regarder les min/max/mean pour avoir une idée des données
- regarder les répartitions de catégorie (équilibre de classe)
- chercher les lignes contenant des nulles ou le nombre de nulle par colonne
- chercher des valeurs aberrantes

Fonctions pandas utiles nettoyage

read_csv

df.info()

df.describe()

df.isnull().sum()

df.drop(column=[“ “,” “] , inplace = True)

df.colonne.value_count

to_csv

df.fillna

df.dropna

df.colonne.unique

df.get_dummies

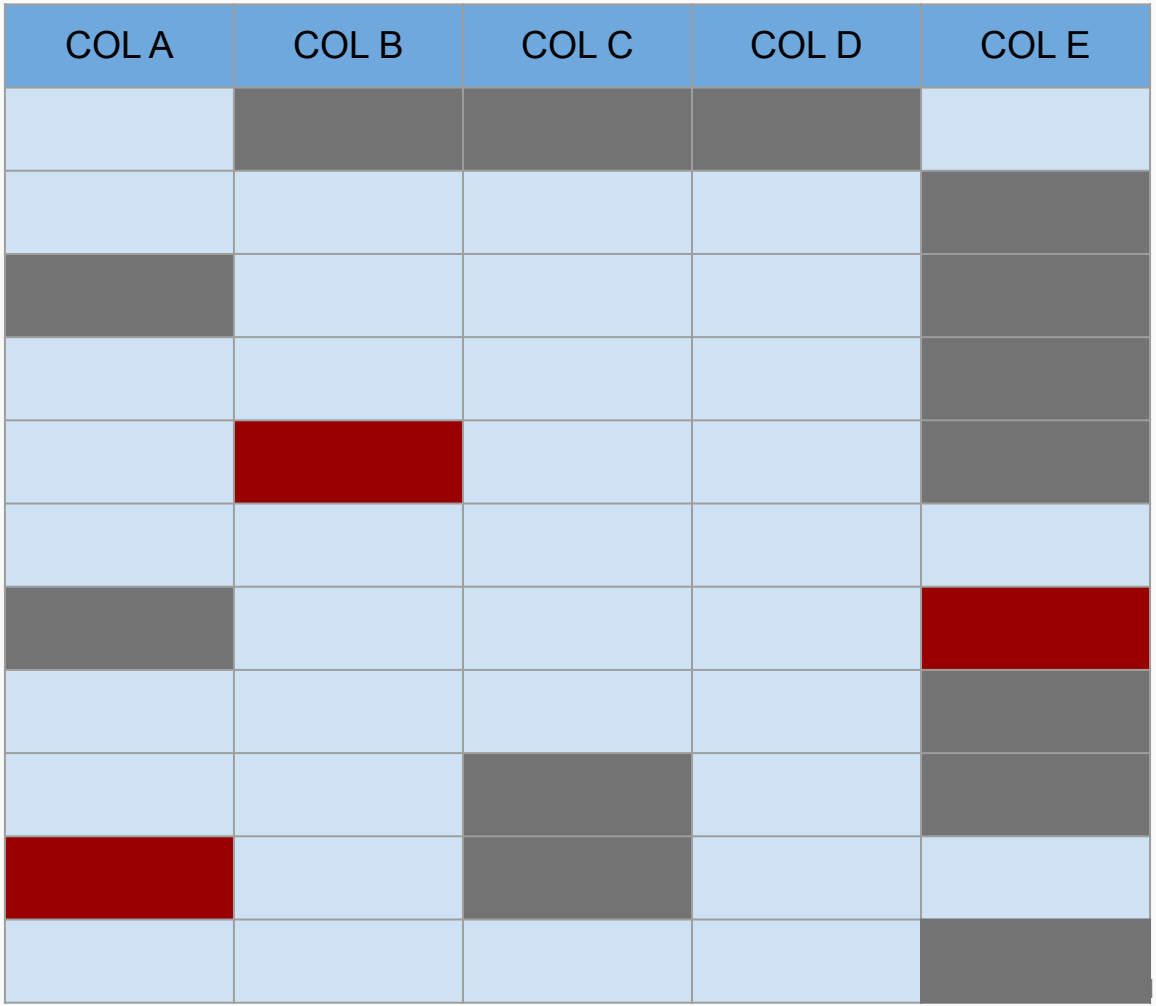
Trouver les valeurs nulles ou aberrantes

Les valeurs aberrantes sont des points qui ne devraient pas exister:

- pixel négatif
- valeur délirante
- un caractère à la place d'un chiffre

En gris des valeurs Nulles
En bleue des valeurs normales
En rouge des valeurs aberrantes

Que fait on?



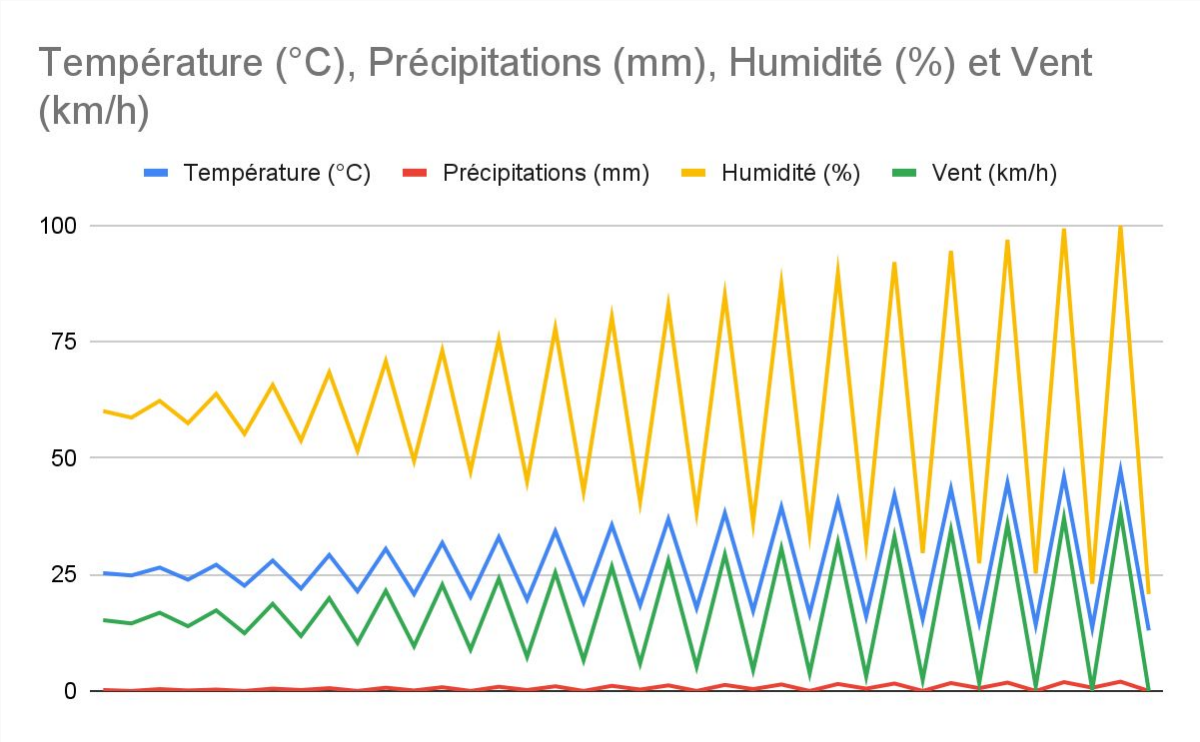
Normaliser les données

Pourquoi les normaliser?

Comment les normaliser?

Transformer les données

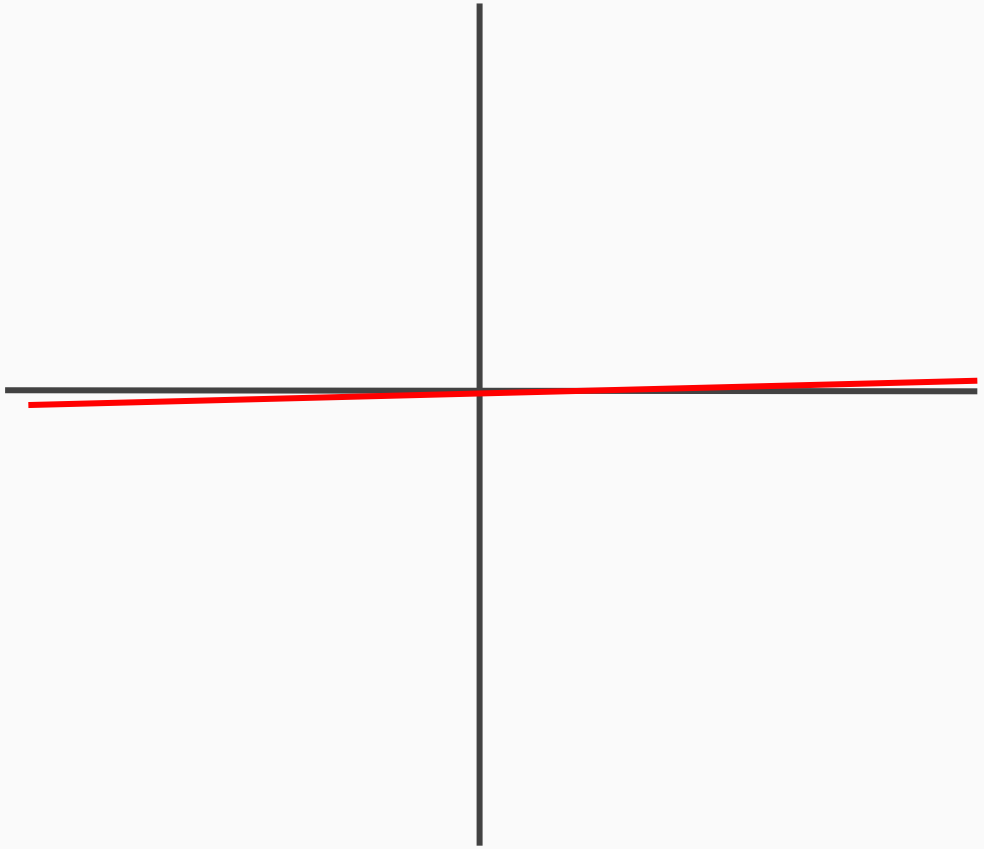
Limitation de vitesse à 33.5 km/h ?



Droite Affine

encore une fois un fond bleu et apaisant

$y1 = a1\ x + b1$

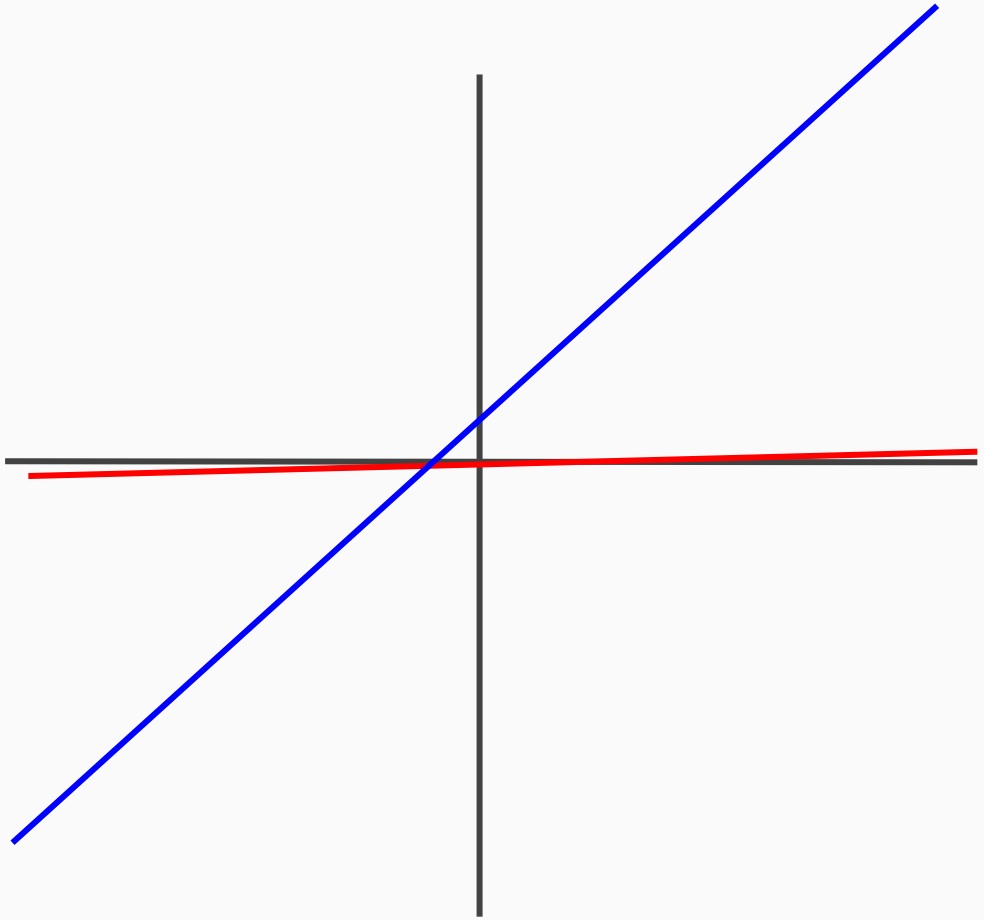


Droite Affine

encore une fois un fond bleu et apaisant

$y1 = a1\ x + b1$

$y2 = a2\ x + b2$

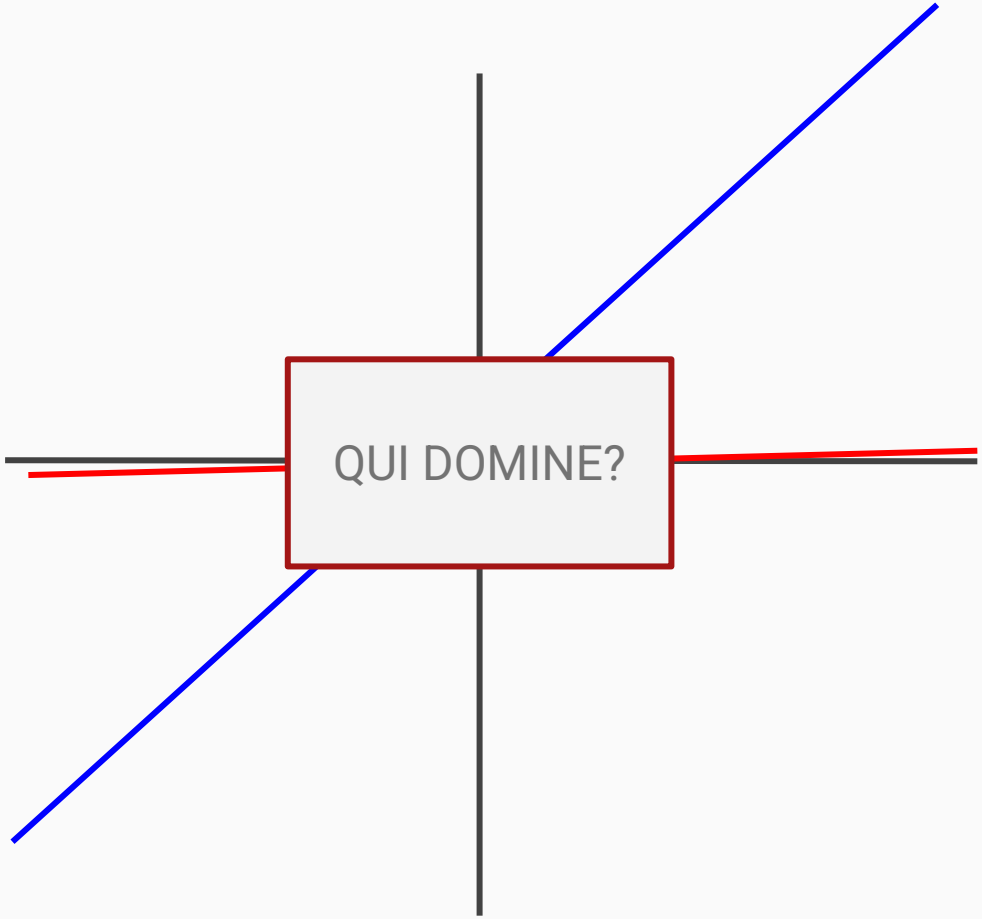


Droite Affine

encore une fois un fond bleu et apaisant

$y1 = a1\ x + b1$

$y2 = a2\ x + b2$



data visualisation

La data visualisation est un art, mais aussi une science !

3 questions clés à se poser

La visualisation la plus efficace n'est pas forcément la plus complexe.

L'objectif est de faciliter la compréhension de vos données.

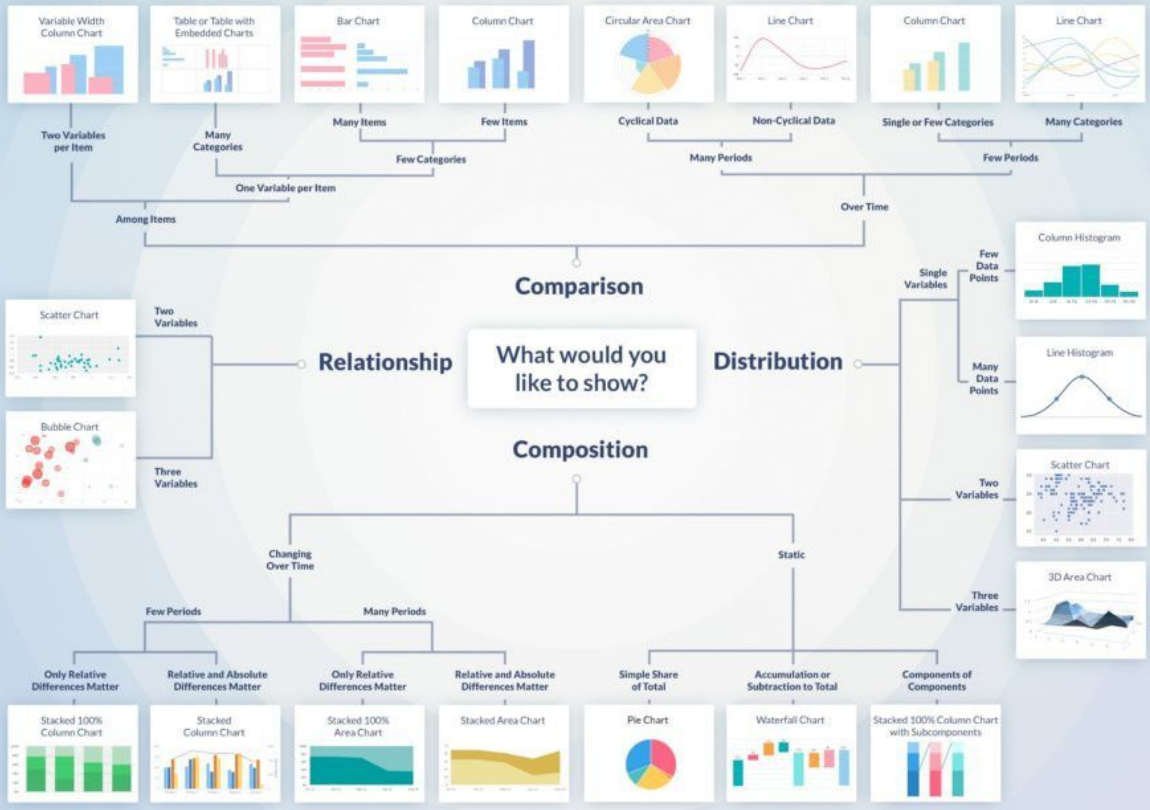
1) Quel est votre objectif ?
- Comparaison
- Relation
- Distribution
- Composition

2) Quelle est la nature de vos données ?
- Variables uniques/multiples,
- périodes
- catégories

3) Quel message souhaitez-vous faire passer ?

Quelques règles d'or :
- comparer des éléments : graphiques en barres/colonnes
- montrer une évolution : des courbes
- des proportions: les camemberts ou graphiques en aires
- des corrélations : le nuage de points

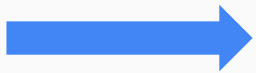
Guided Visualizations for Charts and Graphs



Transformer les données catégorielle

Exemple: chien, chat et poulpe

Animal
Chien
Chat
Poulpe



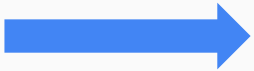
Animal
0
1
2

Transformer les données catégorielle

Exemple: chien, chat et poulpe

Des fois ça marche et des fois ça ne marche pas.

Animal
Chien
Chat
Poulpe



Animal
0
1
2

Poulpe = 2 chats ?

Chat - chat = Chien ?

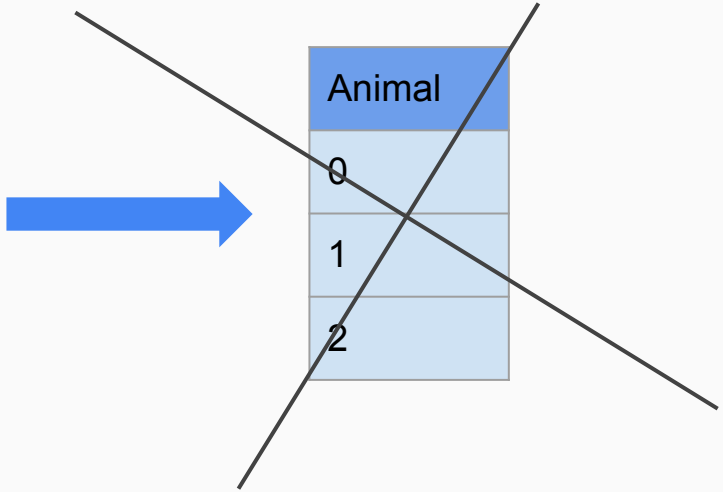
2 chats = 1 poulpe ?



Transformer les données catégorielle

Exemple: chien, chat et poulpe

Animal
Chien
Chat
Poulpe



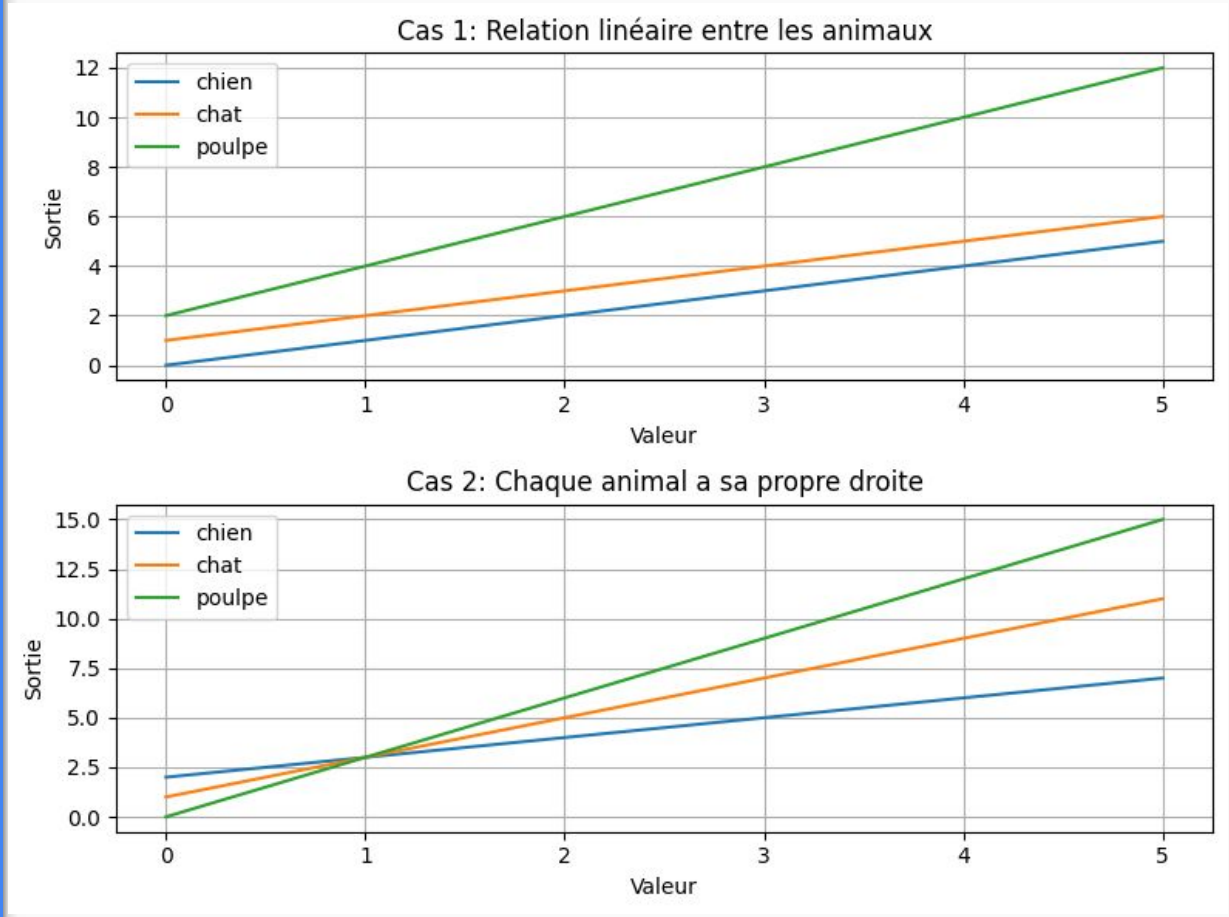
Animal
0
1
2

Animal Chien	Animal Chat	Animal Poulpe
1	0	0
0	1	0
0	0	1

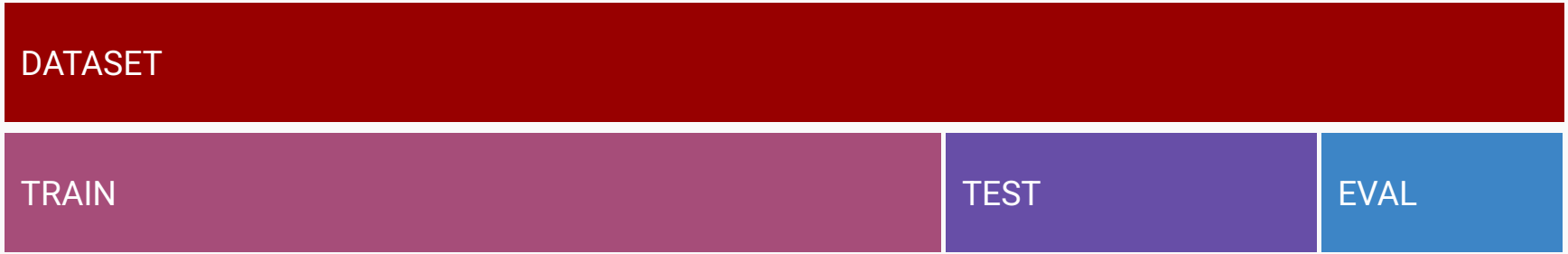
En terme de maths

Si on se place dans le cas où les données ont une relation linéaire avec la sortie alors:

- Le premier cas correspond à avoir une droite pour les chats, chiens et poulpe et avoir la relation chat=chien+1 ou poulpe = 2 chats
- Dans le deuxième cas, le chat, le chien et le poulpe ont chacun leur propre paramètres pour leur droite



Train / Test & Eval OR Train / Valid & Test



TRAIN → Entraîner le modèle

TEST → Évaluer le modèle / Fine Tuning

EVAL → Mesure de performance du modèle

Live Coding

```
df_titanic = sns.load_dataset('titanic')
df_iris = sns.load_dataset('iris')
```

\+ Exercices

```
df = sns.load_dataset('penguins')
df = datasets.fetch_openml(name='adult', version=2)
msno.matrix(df.frame)
```

Veille Techno

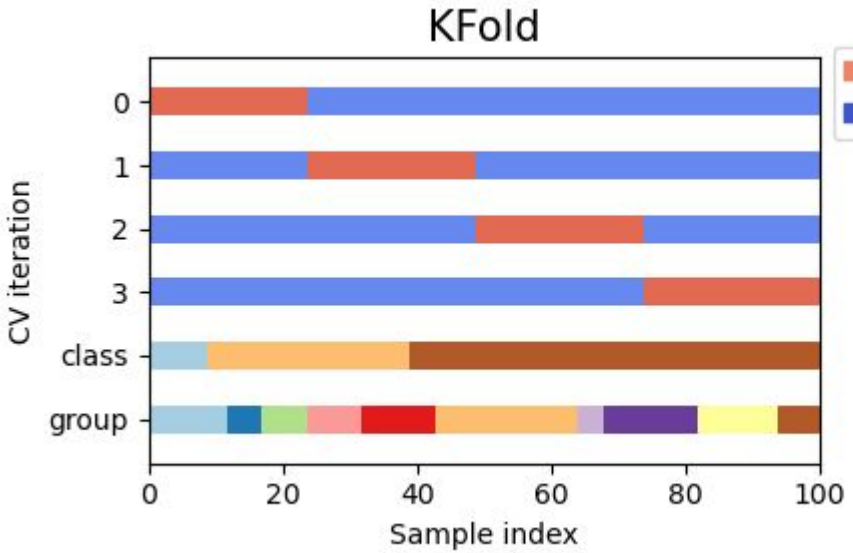
Lire les définitions et les expliquer avec ses propres mots

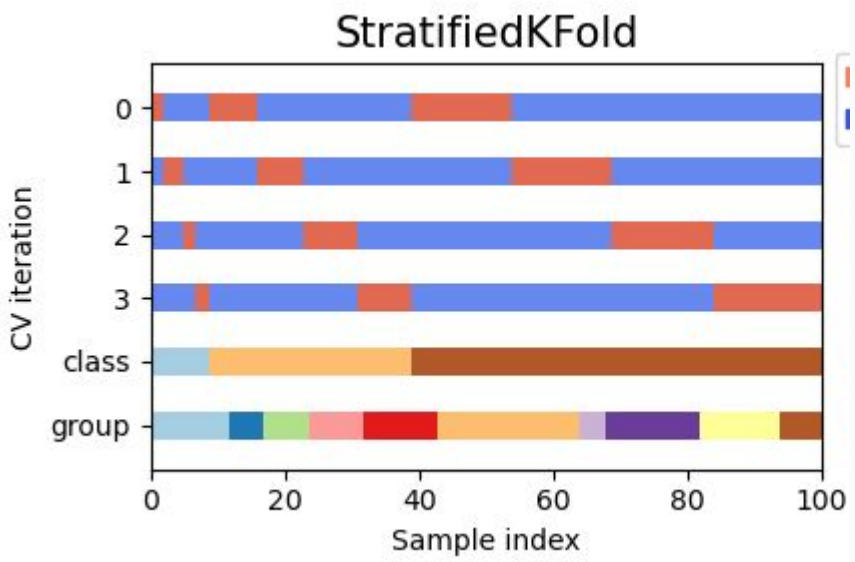
Citer les sources

faire un petit exemple en expliquant les étapes

- split & fold
- fine tuning & feature importance
- Mesure de performance & overfitting
  - Confusion Matrix,
  - Accuracy,
  - Recall,
  - F1-score,
  - unbalanced Accuracy
  - overfitting,
  - ROC AUC
- bagging & boosting
- Arbre de décision
- SVM

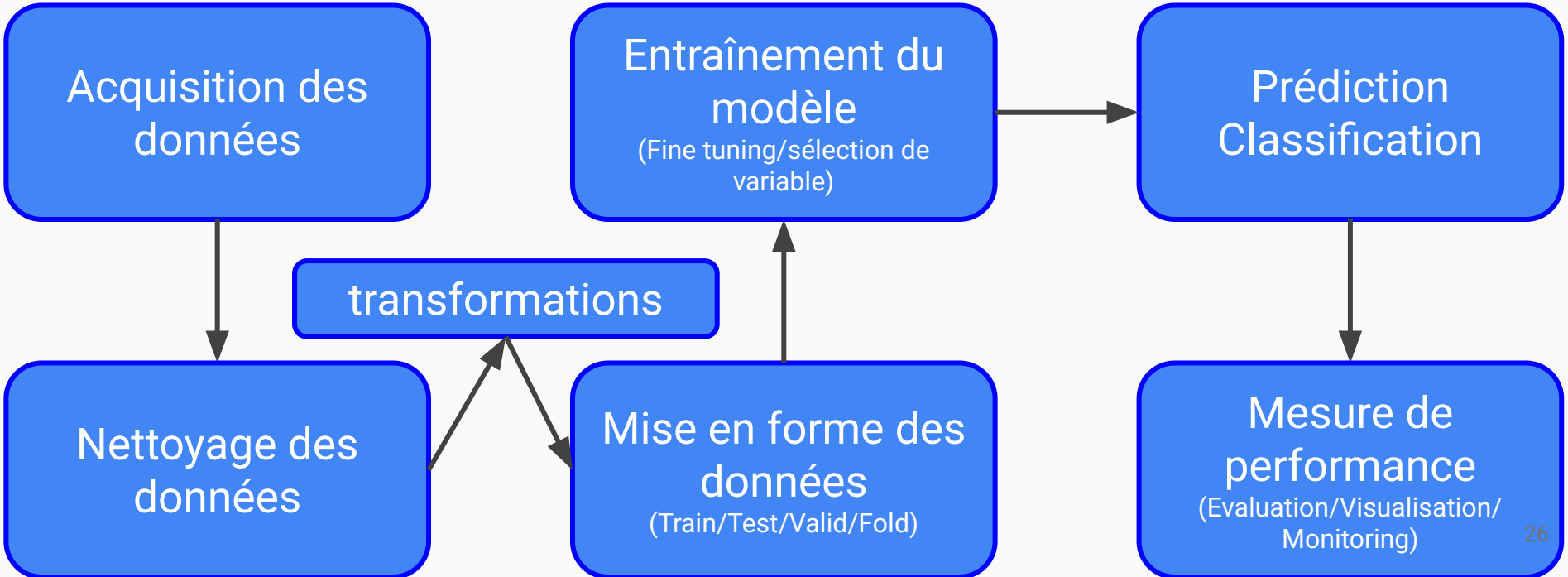
Cross Validation - Fold





[3.1. Cross-validation: evaluating estimator performance]

Pipeline Machine Learning (de base)



Veille Pipeline/Column transformer

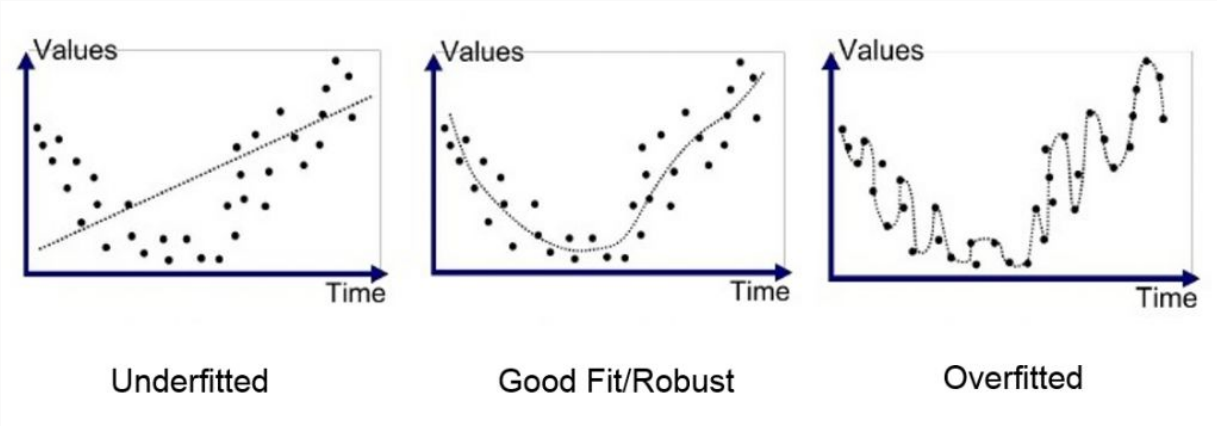
```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
numerical_features = df.select_dtypes(include=['int64', 'float64']).columns
categorical_features = df.select_dtypes(include=['object', "category"]).columns
numerical_transformer = Pipeline(steps=[ ('scaler', StandardScaler()) ])
categorical_transformer = Pipeline(steps=[ ('encoder', OneHotEncoder()) ])
# Combine the preprocessing steps for numerical and categorical features
preprocessor = ColumnTransformer(transformers=[ ('num', numerical_transformer,
numerical_features), ('cat', categorical_transformer, categorical_features) ])

X = preprocessor.fit_transform(df)
```

Fine Tuning / optimisation hyperparamètres

Évaluer le modèle en utilisant des grilles d'hyper-paramètres et retenir le meilleur modèle

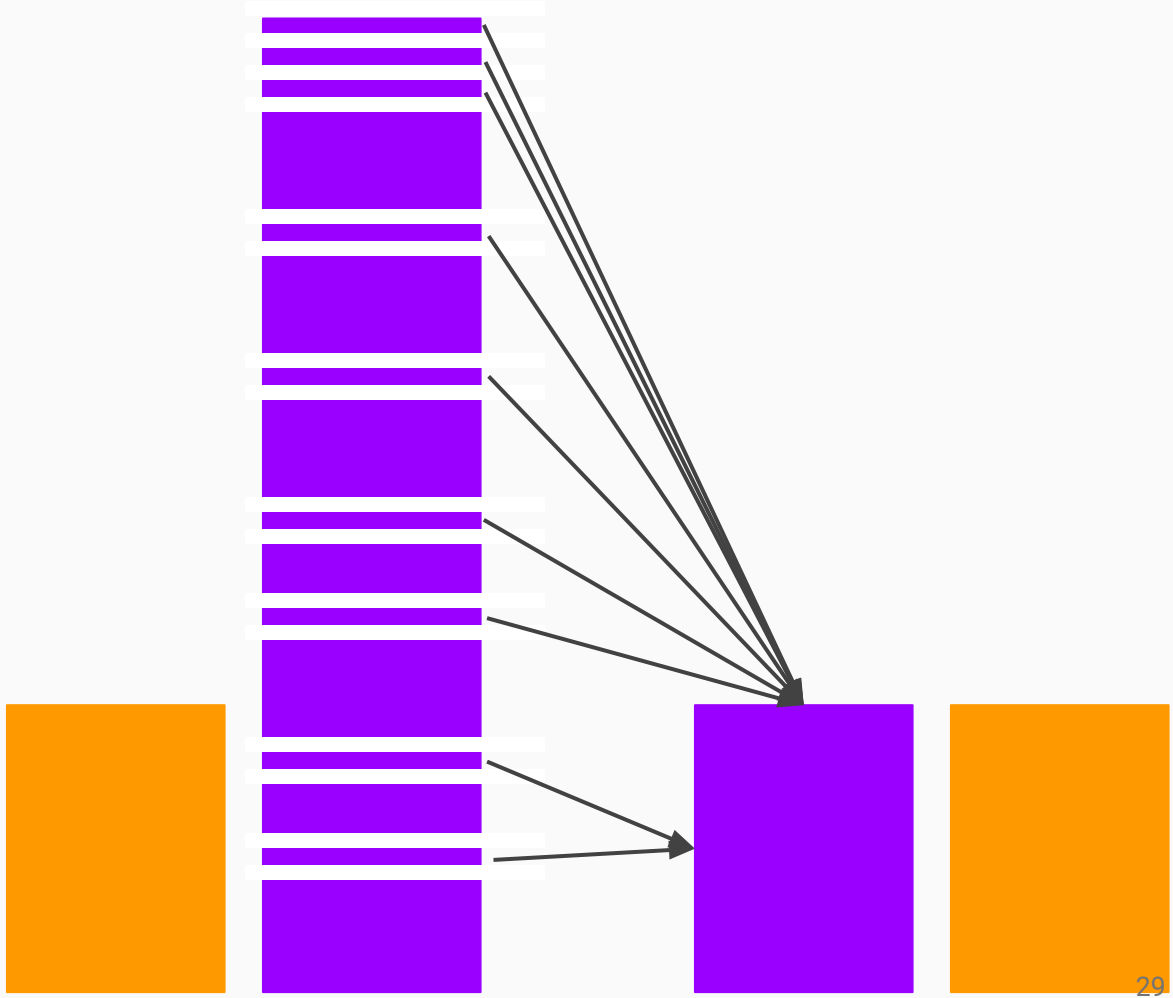
Attention à l'over fitting



GridSearchCV — scikit-learn 1.8.0 documentation

Under Sampling

On jette des échantillons de la classe la plus représentée

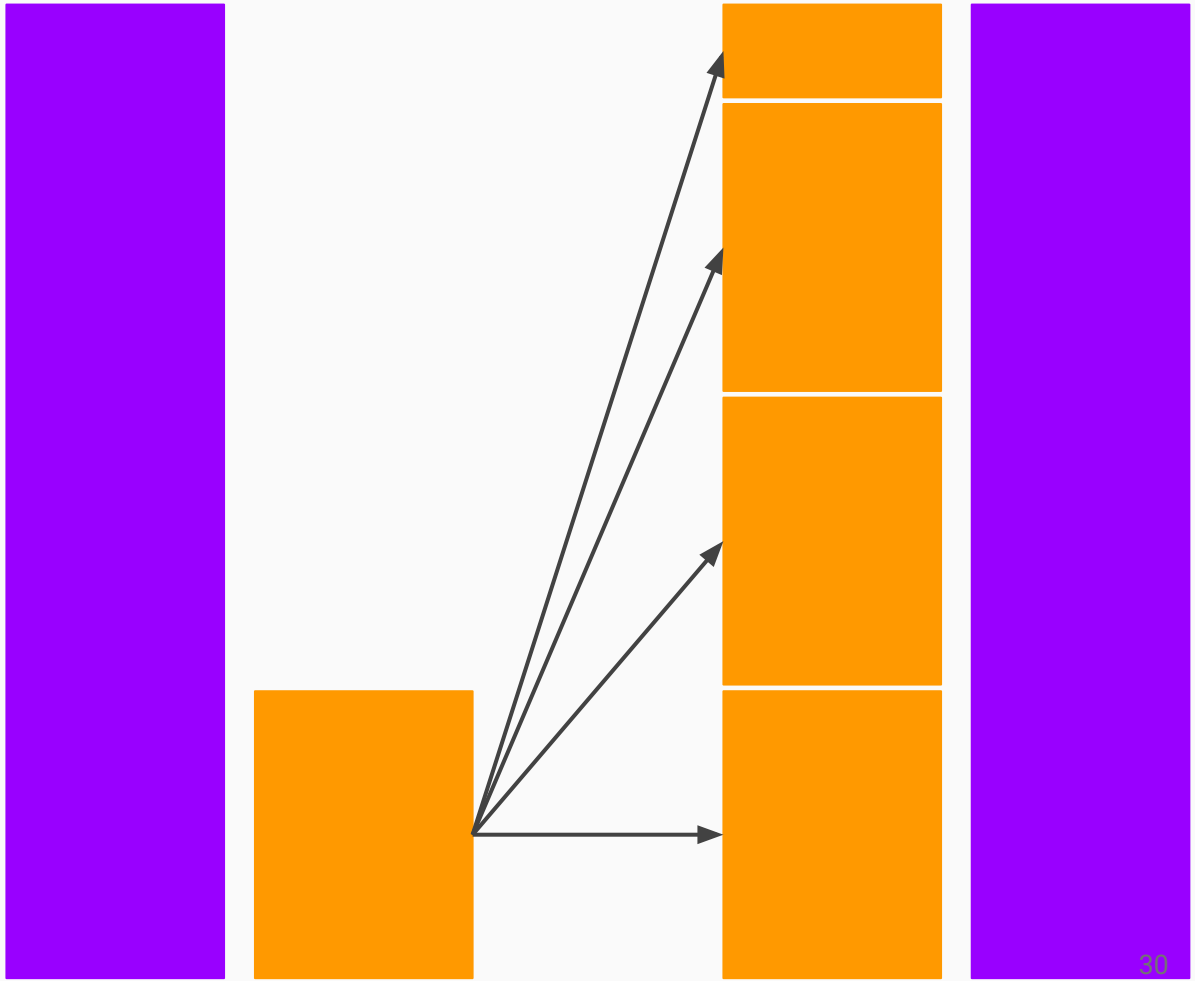


Over Sampling

On répète les échantillons de la classe la moins représentée

Imbalanced learn

SMOTE — Version 0.13.0



Matrice de confusion - dis autrement

	CANARD	PAS CANARD
POSITIF	Tu es un canard 	Tu es un canard 
NÉGATIF	Tu n’es pas un canard 	Tu n’es pas un canard 

Mesure de performance - Classification

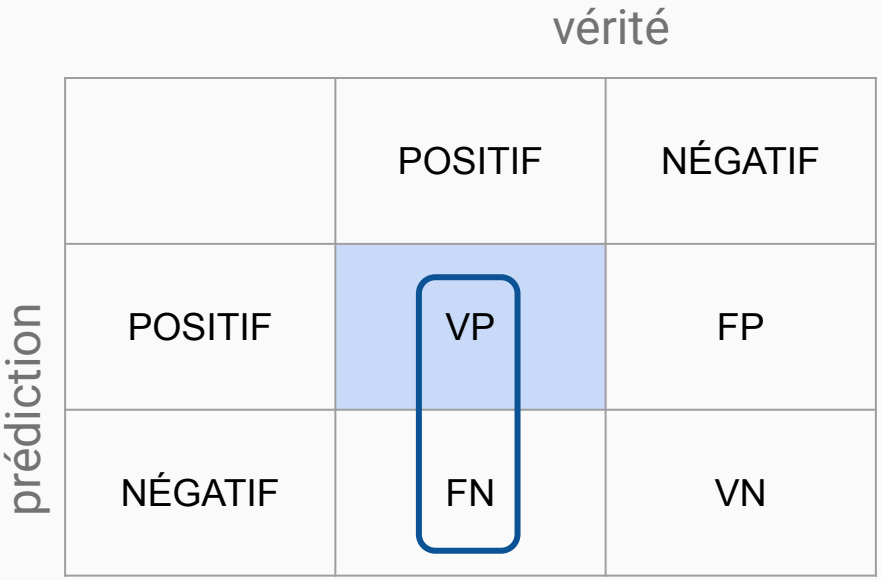
prédiction \ vérité	POSITIF	NÉGATIF
POSITIF	VP	FP
NÉGATIF	FN	VN

Précision = (Vrais positifs) / (Vrais positifs + Faux positifs)

la proportion d'éléments positifs correctement identifiés parmi tous les éléments identifiés comme positifs.

0 = bad
100% good

Mesure de performance - Classification



Rappel= (Vrais positifs) / (Vrais positifs + Faux négatif)

la proportion d'éléments positifs correctement identifiés parmi tous les éléments réellement positifs

0 = bad
100% good

Mesure de performance - Classification

	POSITIF	NÉGATIF
POSITIF	VP	FP
NÉGATIF	FN	VN

Accuracy = (VP+VN) / (VP+FP+FN+VN)

la proportion d'éléments correctement identifiés parmi tous les éléments

0 = bad
100% good

Mesure de performance - Classification

	POSITIF	NÉGATIF
POSITIF	VP	FP
NÉGATIF	FN	VN

f1 score = 2 * (Precision * Recall) / (Precision + Recall)

~~la proportion d'éléments positifs correctement identifiés parmi tous les éléments réellement positifs~~

0 = bad
100% good

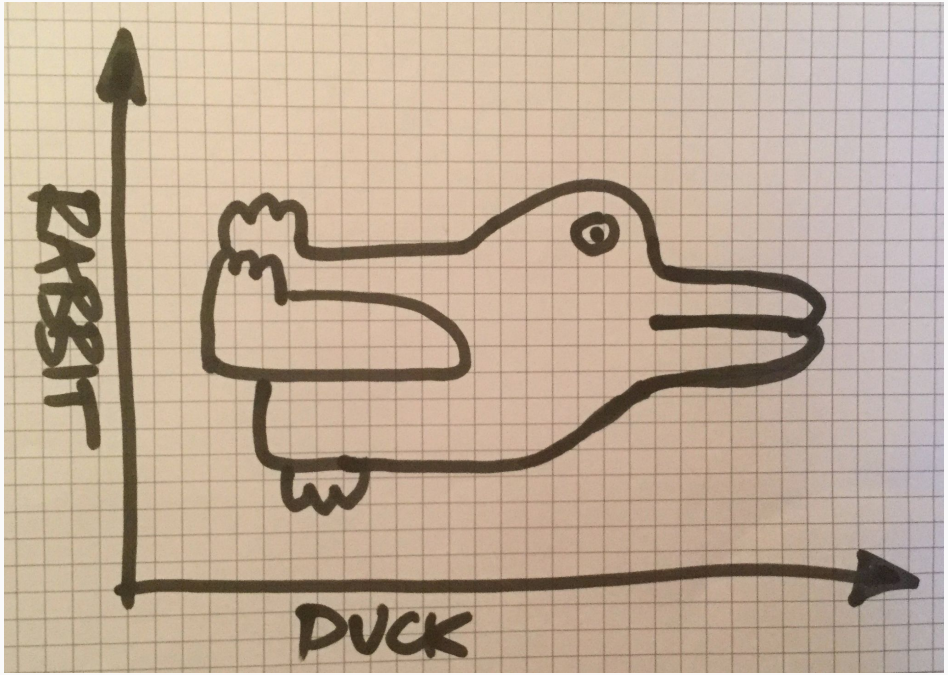
Mesure de performance - Classification

	POSITIF	NÉGATIF
POSITIF	VP	FP
NÉGATIF	FN	VN

Balanced Accuracy: calculé en prenant la moyenne des sensibilités (taux de vrais positifs) de chaque classe. En d'autres termes, il mesure la capacité d'un modèle à prédire correctement toutes les classes, en accordant une attention égale à chacune d'entre elles

$$\text{Balanced Accuracy} = \frac{1}{2}\left(\frac{\text{VP}}{\text{VP+FN}} + \frac{\text{VN}}{\text{VN+FP}}\right)$$

Rabbit Duck



MultiClass Versus MultiLabel

CHIEN	CHAT	POULE
0	0	1
1	0	0

CHIEN	CHAT	POULE
0	1	1
1	0	1

exemple de matrice de confusion délicate Calculez les métriques et analysez

Données médicale

	Réel Malade	Réel Sain
Prédit Malade	8	10
Prédit Sain	2	9990

capteur incendie

	Réel Incendie	Réel Pas d'incendie
Prédit Incendie	90	1
Prédit Pas d'incendie	10	99

Mesure de performance de classification

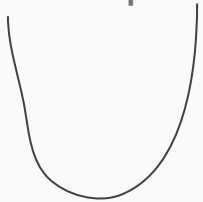
sklearn.metrics

roc_auc_score — scikit-learn 1.8.0 documentation

classification_report — scikit-learn 1.8.0 documentation

Mesure de performance - régression

MSE: Mean Square Error



$$\frac{1}{N}\sum_n^N (\hat{y}_n - y_n)^2$$

MAE: Mean Absolute Error



$$\frac{1}{N}\sum_n^N |\hat{y}_n - y_n|$$

R2, entre 0 (bad) 1 (good)

$$1- \frac{\frac{1}{N}\sum_n^N (\hat{y}_n - y_n)^2}{\frac{1}{N}\sum_n^N (y_n - \mu)^2} = \frac{\frac{1}{N}\sum_n^N (\hat{y}_n - y_n)^2}{\sigma^2}$$

Exemple pour la régression

Prix Réel (y)	Prédit Modèle A	Erreur A	Prédit Modèle B	Erreur B
200 000	210 000	+10 000	200 000	0
250 000	240 000	-10 000	250 000	0
300 000	310 000	+10 000	300 000	0
150 000	140 000	-10 000	150 000	0
200 000	210 000	+10 000	**250 000**	**+50 000**

mse A 100 * 10**6

mae A 10 * 10**3

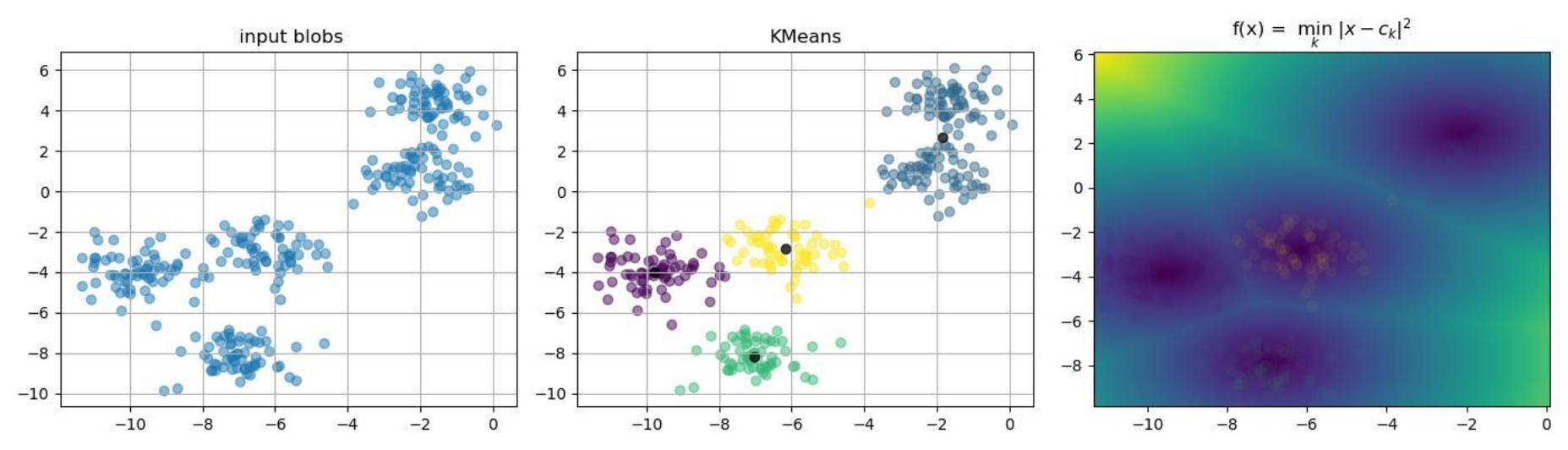
mse B 500 * 10**6

mae B 10 * 10**3

Calculer le MSE et le MAE pour les deux modèles (A et B)

Apprentissage non supervisé

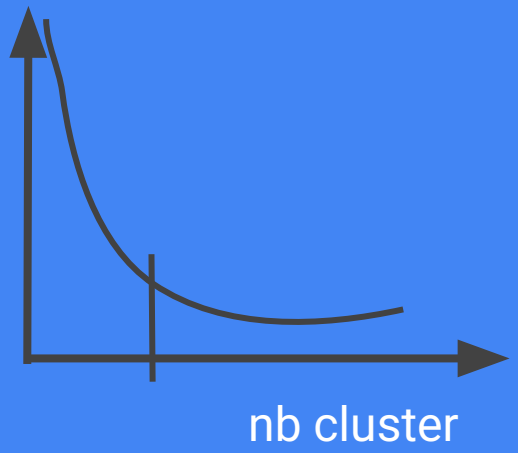
Clustering



exemple

Kmeans

méthode du coude (elbow)




```python
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

import numpy as np
import matplotlib.pyplot as plt

#%%
np.random.seed(1)
X, _ = make_blobs(n_samples=300, centers=5, cluster_std=.8)

#%%
NBClust = 4
# kmeansclust with K=2
kmeansclust = KMeans(n_clusters=NBClust)
# fit the model
kmeansclust.fit_predict(X)
# distance from samples to cluster
dist = kmeansclust.transform(X)
# get the label
label = kmeansclust.labels_
# get the centroids
C = kmeansclust.cluster_centers_
```

Pourquoi le monitoring est crucial ?

Votre modèle de machine learning n'est pas une île.

- **Les données changent :** Le monde réel évolue, votre modèle doit s'adapter.
- **Les modèles dérivent :** Les performances se dégradent sans que vous le sachiez.
- **Les performances réelles comptent :** Ce qui fonctionne en test ne fonctionne pas toujours en production.
- **L'amélioration est continue :** Le monitoring vous guide vers un modèle meilleur.
- **La confiance se mérite :** Un modèle non surveillé est un modèle à risque.

MLflow vous donne les outils pour garder le cap dans cet océan de données.



Enregistrement des métriques de performance

```python
import mlflow.sklearn
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

# Charger les données MNIST
digits = load_digits()
X = digits.data
y = digits.target

# Diviser les données en ensembles d'entraînement et de test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```python
# Créer un modèle de régression logistique
model = LogisticRegression(max_iter=1000)
# Démarrer un run MLflow
with mlflow.start_run():
    # Entraîner le modèle
    model.fit(X_train, y_train)

    # Faire des prédictions sur l'ensemble de test
    y_pred = model.predict(X_test)

    # Calculer la précision
    accuracy = accuracy_score(y_test, y_pred)

    # Enregistrer les métriques
    mlflow.log_metric("accuracy", accuracy)

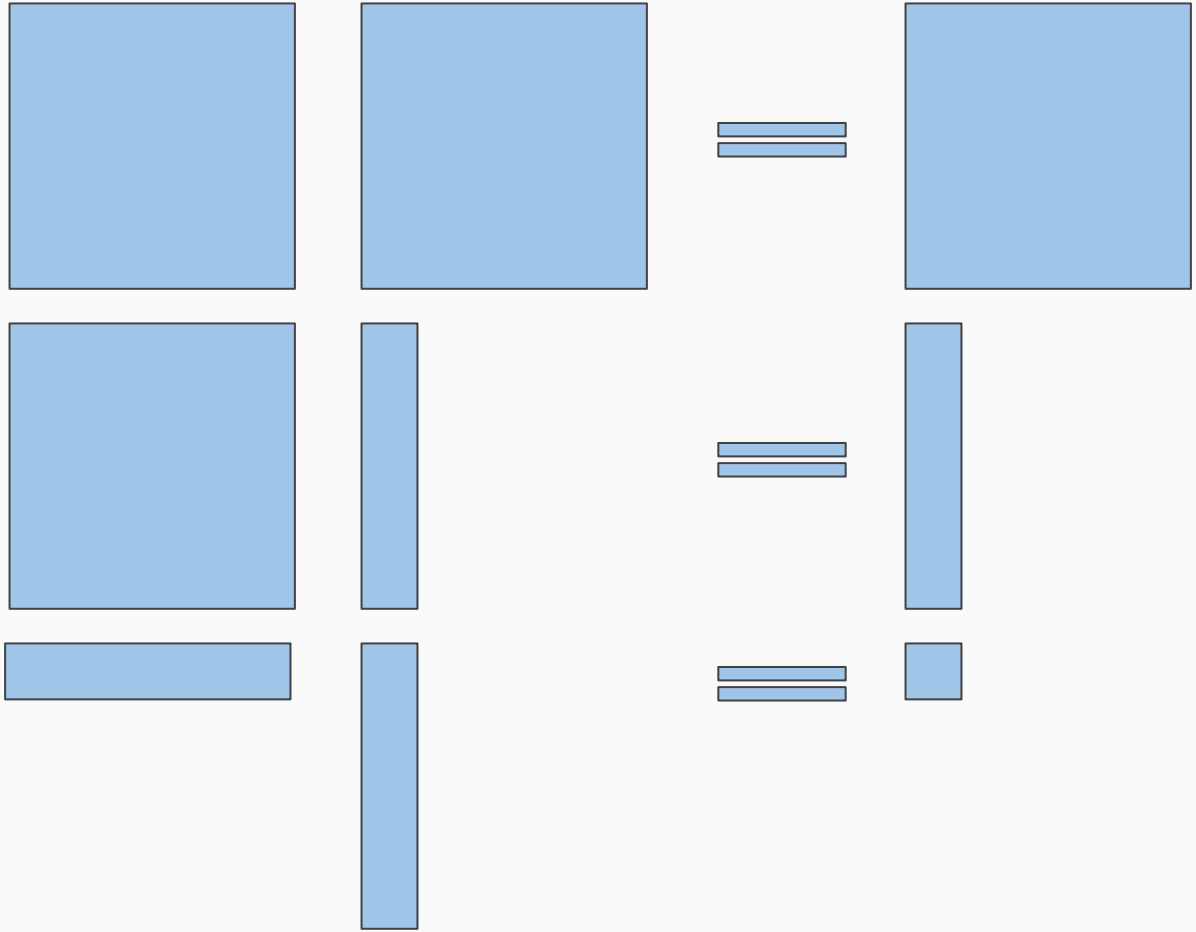
    # Enregistrer le modèle
    mlflow.sklearn.log_model(model, "model")
```

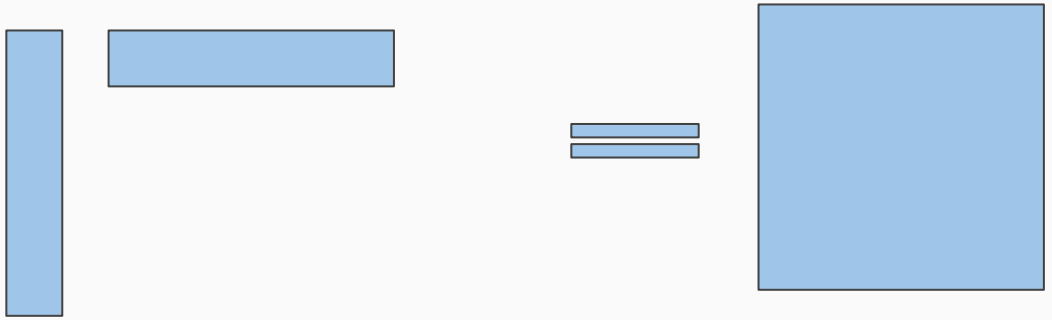
Réduction de dimension

Technique permettant d'expliquer X% des données en réduisant les dimensions à une taille beaucoup plus petite. Intérêt:

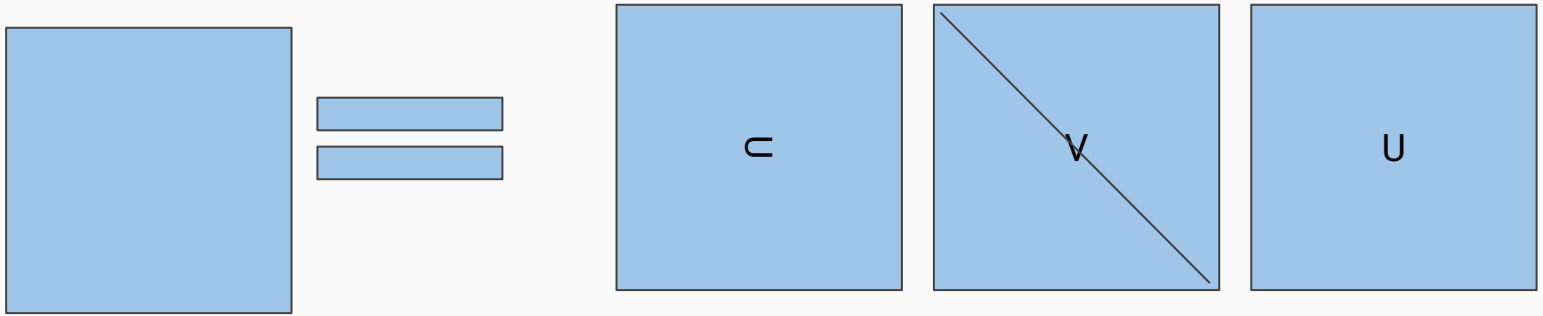
- permet de visualiser la distances entre des données en grande dimension (>3D) sur un plot en 2D
- permet de réduire le bruit
- simplifier les données et donc la charge de calcul
- anonymat des données (RGPD)

.jpg



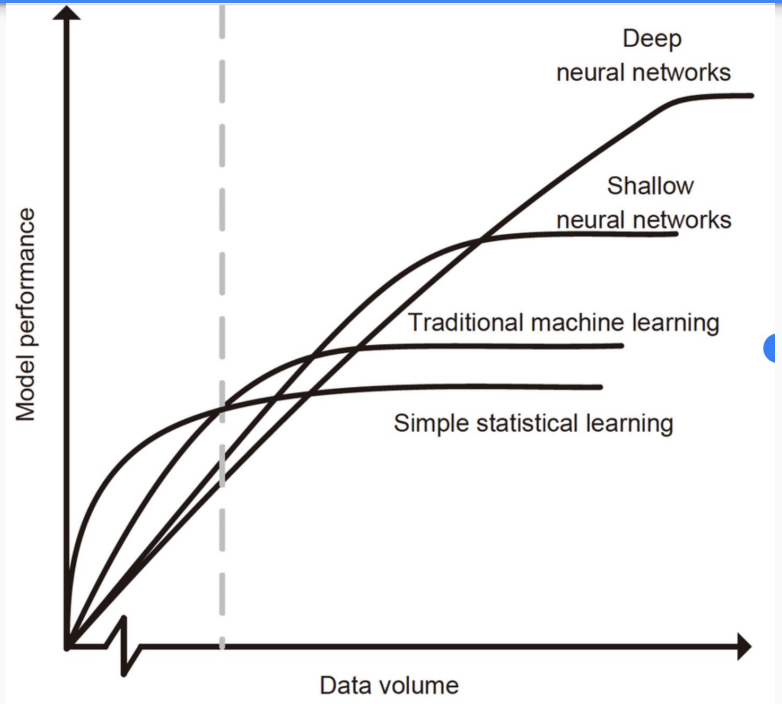


PCA (en) ACP (fr)
Analyse en composante principale



Vecteurs propres et des valeurs propres

ML vs RN


Deep
neural networks
Shallow
neural networks
Traditional machine learning
Simple statistical learning
Model performance
Data volume